

Tham lam (Greedy)

Nội dung

1. Tham lam là gì?
 - Ví dụ đời thường: Đổi tiền
 - Tại sao gọi là "tham lam"?
2. Khi nào tham lam đúng, khi nào sai?
 - Khi tham lam ĐÚNG:
 - Khi tham lam SAI — Ví dụ phản chứng:
 - Bài học rút ra:
3. Bài toán kinh điển: Activity Selection (Chọn hoạt động)
 - Đề bài
 - Ví dụ
 - Thử các chiến lược tham lam khác nhau
 - Tại sao "kết thúc sớm nhất" là chiến lược đúng?
 - Code hoàn chỉnh (Movie Festival — CSES 1629):
4. Dạng "Sort rồi xử lý" (Sort-then-Greedy)
 - Ví dụ: Hiệu hai số nhỏ nhất
 - Ví dụ: Ghép cặp với chênh lệch tối đa
5. Kỹ thuật chứng minh: Exchange Argument (Lập luận hoán đổi)
 - Ý tưởng
 - Ví dụ đơn giản
6. Bài toán mẫu hoàn chỉnh: Missing Coin Sum (CSES 2183)
7. Các dạng tham lam phổ biến
8. Sai lầm thường gặp và mẹo
 - Sai lầm 1: Giả định tham lam đúng mà không suy nghĩ kỹ
 - Sai lầm 2: Sort sai tiêu chí
 - Sai lầm 3: Quên edge case
 - Mẹo: Nếu không chắc, thử brute force trước
9. Tổng kết

Chào mừng bạn đến với bài học về thuật toán tham lam (Greedy)! Đây là một trong những kỹ thuật "đẹp" nhất trong lập trình thi đấu — code thường ngắn gọn, chạy nhanh, nhưng đòi hỏi tư duy sắc bén để nhận ra lựa chọn đúng tại mỗi bước.

1. Tham lam là gì?

Thuật toán tham lam là chiến lược giải bài toán bằng cách: tại mỗi bước, đưa ra lựa chọn tốt nhất lúc đó (lựa chọn "tham lam") mà không nhìn lại, không xem xét các bước tương lai.

Ví dụ đời thường: Đổi tiền

Bạn cần trả 37.000đ bằng các tờ tiền mệnh giá 20.000đ, 10.000đ, 5.000đ, 2.000đ, 1.000đ. Bạn muốn dùng ít tờ nhất.

Cách bạn làm tự nhiên:

1. Lấy 1 tờ 20.000đ → còn 17.000đ
2. Lấy 1 tờ 10.000đ → còn 7.000đ
3. Lấy 1 tờ 5.000đ → còn 2.000đ
4. Lấy 1 tờ 2.000đ → còn 0đ

Tổng: 4 tờ. Đây chính là chiến lược tham lam: luôn chọn tờ tiền lớn nhất có thể.

Tại sao gọi là "tham lam"?

Vì thuật toán luôn "tham"— luôn lấy cái tốt nhất ngay lập tức, không cần suy nghĩ lâu dài. Giống như một người tham ăn: thấy miếng nào ngon nhất thì lấy trước!

2. Khi nào tham lam đúng, khi nào sai?

Câu hỏi quan trọng nhất: Không phải lúc nào tham lam cũng cho đáp án đúng!

Khi tham lam ĐÚNG:

Với hệ tiền tệ thông thường (1, 2, 5, 10, 20, 50, 100, 200, 500...), lấy tờ lớn nhất luôn cho đáp án tối ưu. Đây là nhờ tính chất đặc biệt: mỗi mệnh giá lớn hơn đều là bội hoặc gần bội của mệnh giá nhỏ hơn.

Khi tham lam SAI — Ví dụ phản chứng:

Giả sử các mệnh giá là: 1đ, 3đ, 4đ. Cần trả 6đ.

Tham lam: Lấy tờ lớn nhất trước:

- 4đ → còn 2đ
- 1đ → còn 1đ
- 1đ → còn 0đ
- Tổng: 3 tờ (4 + 1 + 1)

Tối ưu: 3đ + 3đ = 2 tờ!

Tham lam SAI vì lấy tờ 4đ đã "đóng cánh cửa" dẫn đến nghiệm tốt hơn. Để giải đúng bài đổi tiền tổng quát, ta cần quy hoạch động (DP) — bạn sẽ học ở bài sau.

Bài học rút ra:

Tham lam chỉ đúng khi ta có thể CHỨNG MINH nó đúng. Nếu không chắc, hãy dùng DP hoặc brute force.

3. Bài toán kinh điển: Activity Selection (Chọn hoạt động)

Đây là bài toán tham lam nổi tiếng nhất, xuất hiện trong hầu hết mọi sách giáo khoa.

Đề bài

Có n bộ phim chiếu tại rạp. Bộ phim thứ i bắt đầu lúc s_i và kết thúc lúc e_i . Bạn muốn xem nhiều bộ phim nhất, nhưng không thể xem hai phim cùng lúc (nếu phim A kết thúc lúc t , bạn có thể xem phim B bắt đầu lúc t).

Ví dụ

Input: 4 phim: (1, 2), (2, 4), (1, 5), (4, 6)

Thời gian: 1 2 3 4 5 6

Phim 1: [==]

Phim 2: [====]

Phim 3: [=====]

Phim 4: [====]

Thử các chiến lược tham lam khác nhau

- **Chiến lược 1:** Chọn phim bắt đầu sớm nhất $\rightarrow$ Phim 1(1-2), rồi Phim 2(2-4), Phim 4(4-6) $\rightarrow$ 3 phim. Nhưng chiến lược này thất bại nếu có phim (0, 100) — phim dài choán hết!
- **Chiến lược 2:** Chọn phim ngắn nhất $\rightarrow$ Phim 1 (dài 1), Phim 2 (dài 2), Phim 4 (dài 2) $\rightarrow$ 3 phim. Cũng có phản ví dụ.
- **Chiến lược 3:** Chọn phim kết thúc sớm nhất $\rightarrow$ Sort theo kết thúc: (1, 2), (2, 4), (1, 5), (4, 6).
 - Chọn (1, 2). $\text{last_end} = 2$.
 - (2, 4): bắt đầu $2 \geq 2 \rightarrow$ chọn! $\text{last_end} = 4$.
 - (1, 5): bắt đầu $1 < 4 \rightarrow$ bỏ.
 - (4, 6): bắt đầu $4 \geq 4 \rightarrow$ chọn! $\text{last_end} = 6$.

Kết quả: 3 phim ✓

Tại sao "kết thúc sớm nhất" là chiến lược đúng?

Trực giác: Phim kết thúc sớm nhất "tốn" ít thời gian nhất $\rightarrow$ để lại nhiều thời gian nhất cho các phim còn lại $\rightarrow$ ta có thể chọn thêm nhiều phim hơn.

Chứng minh (tóm tắt): Giả sử nghiệm tối ưu O^* không chọn phim kết thúc sớm nhất f . Phim đầu tiên trong O^* kết thúc muộn hơn f . Ta thay phim đầu tiên đó bằng $f \rightarrow$ số phim không đổi, nhưng thời gian kết thúc sớm hơn $\rightarrow$ không ảnh hưởng các phim sau $\rightarrow$ nghiệm mới cũng tối ưu.

Code hoàn chỉnh (Movie Festival — CSES 1629):

C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<pair<int,int>> movies(n); // (end, start)
    for (int i = 0; i < n; i++) {
        int s, e;
        cin >> s >> e;
        movies[i] = {e, s}; // Lưu end trước để sort theo end
    }
    sort(movies.begin(), movies.end()); // Sort theo end tăng dần
    int count = 0, last_end = -1;
    for (auto &[e, s] : movies) {
        if (s >= last_end) { // Phim này bắt đầu sau khi phim trước kết thúc
            count++;
            last_end = e;
        }
    }
    cout << count << endl;
}
```

Python

```
n = int(input())
movies = []
for _ in range(n):
    s, e = map(int, input().split())
    movies.append((e, s)) # (end, start)
movies.sort() # sort theo end tăng dần
count = 0
last_end = -1
for e, s in movies:
    if s >= last_end:
        count += 1
        last_end = e
print(count)
```

4. Dạng "Sort rồi xử lý" (Sort-then-Greedy)

Rất nhiều bài tham lam bắt đầu bằng bước sắp xếp. Sau khi sort, ta xử lý từ trái sang phải theo một chiến lược nào đó.

Ví dụ: Hiệu hai số nhỏ nhất

Đề bài: Cho mảng a gồm n số. Tìm hiệu nhỏ nhất giữa hai phần tử bất kỳ (tức $\min |a_i - a_j|$ với $i \neq j$).

Phân tích:

- Nếu duyệt tất cả cặp: $O(n^2) \rightarrow$ quá chậm với n lớn.
- Nhận xét: Sau khi sắp xếp, hiệu nhỏ nhất chắc chắn nằm ở hai phần tử liền kề!
- Tại sao? Nếu $a \leq b \leq c$, thì $c - a \geq c - b$ và $c - a \geq b - a$. Vậy hiệu $c - a$ không bao giờ nhỏ hơn hiệu hai phần tử liền kề.

C++

```
sort(a.begin(), a.end());
int min_diff = INT_MAX;
for (int i = 1; i < n; i++)
    min_diff = min(min_diff, a[i] - a[i-1]);
```

Python

```
a.sort()
min_diff = float('inf')
for i in range(1, n):
    min_diff = min(min_diff, a[i] - a[i - 1])
```

Độ phức tạp: $O(n \log n)$ cho sort + $O(n)$ cho duyệt = $O(n \log n)$.

Ví dụ: Ghép cặp với chênh lệch tối đa

Đề bài: Có n học sinh cần đi taxi. Mỗi taxi chở tối đa 2 người. Hai người chở chung phải có chiều cao chênh lệch không quá k . Tìm số taxi ít nhất.

Chiến lược tham lam:

1. Sắp xếp theo chiều cao.
2. Duyệt từ trái sang phải. Nếu hai người liên tiếp chênh lệch $\leq k \rightarrow$ ghép cặp, nhảy 2. Ngược lại $\rightarrow$ người hiện tại đi 1 mình, nhảy 1.

Tại sao đúng? Nếu người i có thể ghép với người $i + 1$ (liền kề sau khi sort), thì đó là lựa chọn tốt nhất vì ghép với người xa hơn ($i + 2, i + 3, \dots$) sẽ "lãng phí"— người $i + 1$ mất cơ hội ghép cặp.

C++

```
sort(h.begin(), h.end());
int taxis = 0;
for (int i = 0; i < n; ) {
    if (i + 1 < n && h[i + 1] - h[i] <= k) {
        i += 2; // ghép cặp
    } else {
        i += 1; // đi một mình
    }
    taxis++;
}
```

Python

```
h.sort()
taxis = 0
i = 0
while i < n:
    if i + 1 < n and h[i + 1] - h[i] <= k:
        i += 2 # ghép cặp
    else:
        i += 1 # đi một mình
    taxis += 1
```

5. Kỹ thuật chứng minh: Exchange Argument (Lập luận hoán đổi)

Đây là kỹ thuật chứng minh phổ biến nhất cho thuật toán tham lam.

Ý tưởng

1. Giả sử có một nghiệm tối ưu O^* khác với nghiệm tham lam G tại một bước nào đó.
2. Chỉ ra rằng: hoán đổi lựa chọn của O^* thành lựa chọn của G tại bước đó $\rightarrow$ nghiệm không tệ hơn.
3. Lặp lại cho tất cả các bước khác nhau $\rightarrow$ biến O^* thành G mà không tệ hơn $\rightarrow G$ cũng tối ưu.

Ví dụ đơn giản

Bài toán: Cho dãy a . Sắp xếp lại dãy để tổng $|a_1 - a_2| + |a_2 - a_3| + \dots + |a_{n-1} - a_n|$ nhỏ nhất.

Tham lam: Sắp xếp dãy tăng dần.

Chứng minh bằng exchange argument:

- Giả sử trong nghiệm tối ưu O^* , có vị trí i mà $O^*[i] > O^*[i+1]$ (hai phần tử liền kề ngược thứ tự).
- Hoán đổi $O^*[i]$ và $O^*[i+1]$.
- Có thể chứng minh rằng tổng chênh lệch không tăng.
- Lặp lại (giống Bubble Sort) $\rightarrow$ biến O^* thành dãy tăng dần $\rightarrow$ dãy tăng dần là tối ưu.

Nói đơn giản: "Nếu hoán đổi bất kỳ hai phần tử nào để gần với thứ tự tham lam hơn mà đáp án không tệ hơn, thì thứ tự tham lam là tối ưu."

6. Bài toán mẫu hoàn chỉnh: Missing Coin Sum (CSES 2183)

Đề bài: Cho n đồng xu với mệnh giá $c_1, c_2, \dots, c_n$. Tìm tổng nhỏ nhất mà bạn không thể tạo ra từ một tập con các đồng xu.

Input: 5 và 2 9 1 2 7. **Output:** 6.

Phân tích:

1. Sắp xếp các đồng xu tăng dần: $[1, 2, 2, 7, 9]$.
2. Gọi S là tổng lớn nhất mà ta đã có thể tạo ra (tức là mọi giá trị từ 1 đến S đều tạo được).
3. Xét đồng xu tiếp theo có giá trị c :
4. Nếu $c \leq S + 1$: ta có thể tạo mọi giá trị từ 1 đến $S + c$. Cập nhật $S = S + c$.
5. Nếu $c > S + 1$: ta không thể tạo giá trị $S + 1$! Đáp án là $S + 1$.

Mô phỏng với $[1, 2, 2, 7, 9]$:

BƯỚC	c	S TRƯỚC	$c \leq S + 1$?	S SAU
Bắt đầu	—	0	—	0
1	1	0	$1 \leq 0 + 1 \checkmark$	$0 + 1 = 1$
2	2	1	$2 \leq 1 + 1 \checkmark$	$1 + 2 = 3$
3	2	3	$2 \leq 3 + 1 \checkmark$	$3 + 2 = 5$
4	7	5	$7 \leq 5 + 1$? KHÔNG!	Dừng lại

Đáp án: $S + 1 = 5 + 1 = 6$.

Kiểm tra: Với $[1, 2, 2]$, ta tạo được: 1, 2, $1+2=3$, $2+2=4$, $1+2+2=5$. Đúng là 1 đến 5 đều OK, nhưng 6 thì không.

Code hoàn chỉnh:

C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<long long> c(n);
    for (auto &x : c) cin >> x;
    sort(c.begin(), c.end());
    long long S = 0;
    for (int i = 0; i < n; i++) {
        if (c[i] > S + 1) break;
        S += c[i];
    }
    cout << S + 1 << endl;
}
```

Python

```
n = int(input())
c = list(map(int, input().split()))
c.sort()
S = 0
for coin in c:
    if coin > S + 1:
        break
    S += coin
print(S + 1)
```

7. Các dạng tham lam phổ biến

DẠNG	CHIẾN LƯỢC	VÍ DỤ
Activity Selection	Sort theo thời gian kết thúc	Movie Festival, lịch phòng họp
Ghép cặp	Sort rồi ghép nhỏ nhất với lớn nhất	Ferris Wheel, taxi
Xây dựng xâu	Ưu tiên ký tự nhỏ nhất/lớn nhất	Xâu nhỏ nhất, xâu đối xứng
Phân phối tài nguyên	Sort theo nhu cầu, phân phối lần lượt	Apartments, phân chia
Xây dựng số	Chọn chữ số tối ưu tại mỗi vị trí	Số lớn nhất chia hết cho 30

8. Sai lầm thường gặp và mẹo

Sai lầm 1: Giả định tham lam đúng mà không suy nghĩ kỹ

Đây là sai lầm phổ biến nhất! Bạn thấy bài "có vẻ" greedy, nhanh chóng code, nộp → Wrong Answer.

Cách phòng tránh:

- Trước khi code, thử ít nhất 3-5 ví dụ nhỏ bằng tay, bao gồm cả edge case.
- Với n nhỏ, viết brute force kiểm tra xem tham lam có cho cùng đáp án không.
- Cố gắng nghĩ phản ví dụ trước khi code.

Sai lầm 2: Sort sai tiêu chí

Rất nhiều bài tham lam phụ thuộc vào cách sắp xếp. Sort theo tiêu chí sai $\rightarrow$ đáp án sai hoàn toàn.

Ví dụ Activity Selection:

- Sort theo bắt đầu sớm nhất $\rightarrow$ SAI (phản ví dụ: phim dài choán hết)
- Sort theo ngắn nhất $\rightarrow$ SAI (phản ví dụ: phim ngắn nằm giữa hai phim khác)
- Sort theo kết thúc sớm nhất $\rightarrow$ ĐÚNG $\checkmark$

Sai lầm 3: Quên edge case

- Mảng rỗng hoặc chỉ có 1 phần tử
- Tất cả phần tử giống nhau
- Giá trị rất lớn (cần long long)
- Không có nghiệm hợp lệ

Mẹo: Nếu không chắc, thử brute force trước

Với $n \leq 15$, bạn có thể viết brute force (duyệt tất cả hoán vị, tất cả tập con) để tìm đáp án đúng, rồi so sánh với chiến lược tham lam trên nhiều test ngẫu nhiên.

9. Tổng kết

Thuật toán tham lam là công cụ mạnh mẽ nhưng cần dùng cẩn thận:

1. **Nhận diện:** Bài yêu cầu tối ưu (min/max), dữ liệu có thể sort, mỗi bước có lựa chọn "hiển nhiên".
2. **Kiểm tra:** Thử nhiều ví dụ, tìm phản ví dụ.
3. **Chứng minh:** Dùng exchange argument hoặc trực giác rõ ràng.
4. **Cài đặt:** Thường rất ngắn gọn — sort + duyệt.

Nếu tham lam không đúng $\rightarrow$ chuyển sang DP (bài học tiếp theo) hoặc brute force (nếu n nhỏ).